IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2019

EIE PART II: MEng, BEng and ACGI

Corrected Copy

SOFTWARE ENGINEERING 2: OBJECT-ORIENTED SOFTWARE ENGINEERING

Monday, 3 June 10:00 am

Time allowed: 2:00 hours

There are THREE questions on this paper.

Answer ALL questions.
Q1 carries 40% of the marks. Questions 2 and 3 carry equal marks (30% each).

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : S. Ben Ismail
Second Marker(s) : M. Cattafi

© Imperial College London

SOFTWARE ENGINEERING 2: OBJECT ORIENTED SOFTWARE ENGINEERING

1. This is a general question about C++ and Object Oriented Software Engineering.

a) i) Describe in words the software architecture represented in the UML class diagram in Fig. 1.1.

[5]

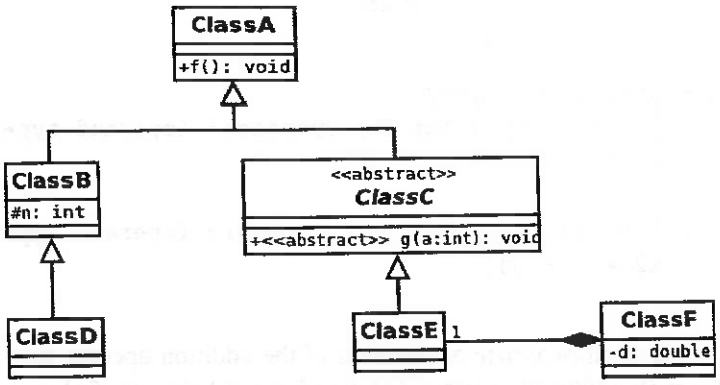


Figure 1.1 A UML class diagram

ii) Write C++ declarations for all the classes in the UML class diagram in Fig. 1.1. The declarations can be kept to the essential skeleton (e.g. constructors can be omitted) but all the elements related to available information (including relationships) should be included.

[7]

b) Explain the difference between the concepts of *Overloading*, *Redefining*, and *Overriding* member functions in C++. Illustrate your answer with an example in C++ code (including a `main()` function calling the overloaded, redefined, and overriden functions).

[12]

c) Consider the binary *addition* (+) operator.

Consider a class `A` having a private member variable `n` of type `int` and a constructor taking one `int` argument.

If `a1` and `a2` are two instances of class `A`, the addition `a1 + a2` can be defined as an object instance of class `A`, which member variable `n` equals the arithmetic addition of `a1.n` and `a2.n`.

i) Write C++ declaration and definition of class `A` overloading the addition operator as a *member function*.

[5]

ii) Write C++ declaration and definition of class `A` overloading the addition operator as a *global function.*

[5]

iii) Consider the following snippet and the two compile-time errors:

```
int main() {
    A a1(0);

    A a2 = a1 + 1;
    a2 = 1 + a1;

    return 0;
}
```

```
$g++ prog.cpp -o prog
$[...] error: no match for operator+ (operand types are A and int)
     A a2 = a1 + 1;
                ^
$[...] error: no match for operator+ (operand types are int and A)
     a2 = 1 + a1;
            ^
```

Provide an appropriate overloading of the addition operator that handles such additions between `int` variables and objects of class `A`, and resolves the two previous compile errors.

(If `a` is an instance of class `A` and `num` is an `int`, the additions `a + num` and `num + a` can be defined as an object instance of class `A`, which member variable `n` equals the arithmetic addition of `a.n` and `num`.)

[6]

2. This question deals with memory management and container classes in C++.

Write the implementation (you can keep the definition and the declaration together) of a container class `intset` representing a *set* of integers, i.e. a data structure in which elements (in this case `int` numbers) are stored without any particular order, and without any repeated values.

Elements can be inserted in an `intset` (making the `intset` grow), and can be removed from it (making it shrink).

a) Declare suitable member data for the class `intset`. The implementation should be based on a dynamically allocated array.

[3]

b) Define a *constructor* which takes as argument an integer representing the initial physical size of the `intset`.

[3]

c) Define the *copy constructor* for the class `intset`.

[5]

d) Define the *assignment operator* for the class `intset`.

[5]

e) Define the *destructor* for the class `intset`.

[2]

f) Define a *contains* member function which returns true if an element (passed as argument) is in the `intset`, and false otherwise.

[2]

g) Define an *insert* member function which inserts an element (passed as argument) in the `intset`. If the element is already in the `intset`, this has no effect.

[4]

h) Overload the *stream insertion* (<<) operator for the class `intset`.

[4]

i) Define an *empty* member function (with no arguments) which returns true if the `intset` is empty and false otherwise.

[1]

j) Define a *size* member function (with no arguments) which returns the number of elements in the `intset`.

[1]

3. This question deals with C++ templates and C++ exceptions.

a) i) Write a template function which, given in input an `std::list` of elements of a generic type (passed by `const` reference), returns the *sum* of its elements. The function should throw a relevant exception if there is no way to return a meaningful sum. The implementation must use iterators.

[8]

ii) Write a `main()` function containing examples of the function being called, including appropriate exception catching.

[6]

iii) Mention an example of a case in which the content of a list passed in input to the function would cause a compiler error, and explain why such error occurs.

[4]

b) Consider the following code.

Trace its execution and its output (including an explanation for all the steps) using the three following definitions of function `f`.

```
#include <iostream>

class my_error {
    public:
        int num;
};


class A {
    public:
        A(int n) {
            if(n == 1) {
                my_error er;
                er.num = 999;
                throw er;
            }
        }
};


void f(); // see three different definitions in i) ii) and iii)


int main() {
    try {
        f();
    }
    catch(my_error er) {
        std::cout << "catch in main: " << er.num << std::endl;
    }
    std::cout << "end of main." << std::endl;

    return 0;
}
```

i) Function f is defined as:

```
void f() {
    try {
        A a(1);
    }
    catch(my_error er) {
        std::cout << "catch in f: " << er.num << std::endl;
    }

    std::cout << "end of f." << std::endl;
}
```

[4]

ii) Function f is defined as:

```
void f() {
    try{
        A a(1);
    }
    catch(my_error er) {
        std::cout << "catch in f: " << er.num << std::endl;
        return;
    }

    std::cout << "end of f." << std::endl;
}
```

[4]

iii) Function f is defined as:

```
void f() {
    A a(1);
}
```

[4]